

GetAlphaBit

الجلسة الثانية

لغة CSS وتنسيق الصفحة

نكسو صفحة الجلسة الماضية — الشكل الآن، لا البنية

2	6
ساعتان	محاور

محاور الجلسة

١	**كيف يصل التنسيق للصفحة**	طرق الربط، وتشريح القاعدة	
٢	**المحدِّدات والأولوية**	من يفوز حين تتعارض القواعد	
٣	**نموذج الصندوق**	كل عنصر صندوق، وهذه طبقاته	
٤	**الوحدات**	متى نستخدم rem ومتى px	
٥	**التدفّق والمواضع**	الخصائص display و position	
٦	**التخطيط بـ Flexbox**	نصف الجلسة هنا — وهو الأهم	

تنبيه على توزيع الوقت

كل ما في CSS يمكن تعلّمه ذاتيًا من المرجع، إلا التخطيط. لذلك نمرّ سريعًا على المحاور الخمسة الأولى، ونعطي المحور السادس نصف الوقت.

في نهاية الجلسة: صفحة السيرة الذاتية نفسها، لكن متجاوبة ومنسّقة.

المحور الأول

كيف يصل التنسيق للصفحة

ثلاث طرق، واحدة منها فقط هي الصحيحة



ثلاث طرق للربط

داخل الوسم

inline

```
<p style="color:red">
```

لا تستعملها: تخلط البنية بالشكل ويصعب تعديلها

داخل الصفحة

internal

```
<style> ... </style>
```

للتجارب السريعة فقط، وداخل ملف واحد

ملف منفصل

external

```
<link rel="stylesheet">
```

الطريقة الصحيحة: ملف واحد يخدم كل الصفحات

```
<link rel="stylesheet" href="style.css">      <!-- inside <head> -->
```

من اليوم: ملف **style.css** واحد، مربوط من الوسم **head**. لا تنسيق داخل وسوم **HTML**.

تشريح القاعدة

```
h1 {
  color: navy;   font-size: 32px;
}
```

المحدِّد

`h1`

من الذي سيتأثّر بهذه القاعدة

كتلة التصريح

`{ ... }`

كل ما بين القوسين ينطبق على المحدِّد

الخاصية

`color`

ما الذي نغيّره في العنصر

القيمة

`navy`

إلى ماذا نغيّره

الفاصلة المنقوطة

`;`

تفصل بين تصريح وآخر — لا تنسها

كل ملف CSS مهما كبر ليس إلا تكرارًا لهذا الشكل الواحد.

المحور الثاني

المحدِّدات والأولوية

من يفوز حين تتعارض قاعدتان



المحدِّدات الأساسية

```
p           { }   /* by element */

.card       { }   /* by class   */

#header     { }   /* by id      */

nav a       { }   /* descendant */

h1, h2, h3  { }   /* grouped    */
```

بالعنصر

كل وسوم هذا النوع في الصفحة

بالصنف

أي عنصر يحمل class بهذا الاسم — الأكثر استعمالًا

بالمعرّف

عنصر واحد فقط، والاسم لا يتكرّر في الصفحة

بالتداخل

الروابط التي تعيش داخل قائمة التنقّل فقط

بالتجميع

قاعدة واحدة تخدم عدة محدِّدات

اعتمد على الصنف class في كل شيء تقريبًا. أمّا المعرّف id فاتركه لجافاسكربت.

الأولوية

١	المحدِّد بالعنصر	`p`	الأضعف
٢	المحدِّد بالصنف	`.card`	أقوى من العنصر
٣	المحدِّد بالمعرّف	`#header`	أقوى من الصنف
٤	التنسيق داخل الوسم	`style=""`	أقوى منها جميعًا
٥	الكلمة important	`!important`	تكسر كل شيء — تجنّبها

عند تساوي القوة، يفوز الأخير في الملف. لهذا ترتيب سطورك مهم.

التتالي والوراثة

الوراثة

```
body { color: #333; }

/* children inherit it */
```

- خصائص النص تُورِّث: اللون والخط والحجم
- خصائص الصندوق لا تُورِّث: الحدود والهوامش
- اضبط الخط مرة واحدة على body

التتالي

```
p { color: red;  }
p { color: blue; }

/* winner: blue */
```

- الأقوى أولًا، فإن تساويا فالأخير يفوز
- لهذا يُوضع ملف المكتبات قبل ملفك
- افتح أدوات المطوّر لترى القاعدة المشطوبة

حين لا تفهم لماذا لا يعمل تنسيقك: افتح F12، وانظر أي قاعدة تشطب قاعدتك.

المحور الثالث

نموذج الصندوق

كل عنصر في الصفحة صندوق، بلا استثناء



نموذج الصندوق

content

النص أو الصورة نفسها

padding

فراغ داخلي بين المحتوى والحد

border

الحدّ المحيط بالعنصر

margin

فراغ خارجي يبعد العنصر عن جيرانه



الهوامش العمودية بين عنصرين تندمج ولا تتجمّع: هامش 20 وهامش 30 يعطيان 30 لا 50.

الخاصية box-sizing

السلوك الافتراضي

```
width: 200px;  padding: 20px;
border: 5px;
-->  real width = 250px
```

العرض الذي كتبته ينطبق على المحتوى فقط، ثم يُضاف إليه الحشو والحد. النتيجة: صناديق أعرض ممّا خطّطت، وتخطيط ينكسر.

السلوك المطلوب

```
box-sizing: border-box;
width: 200px;  padding: 20px;
-->  real width = 200px
```

العرض الذي كتبته هو العرض النهائي فعلًا، بما فيه الحشو والحد. الحساب يصبح بديهيًا ولا ينكسر شيء.

```
* , *::before , *::after { box-sizing: border-box; }
                                /* first line of every project */
```

احفظ هذا السطر. ضعه في أعلى كل ملف CSS تكتبه، ولن تعود لهذه المشكلة أبدًا.

الوحدات

`px`	**بكسل ثابت**	الحدود والظلال وكل ما يجب ألّا يتغيّر
`rem`	**نسبة من خط الجذر**	الخطوط والمسافات — الأكثر أمانًا واستعمالًا
`em`	**نسبة من خط العنصر نفسه**	الحشو داخل زر ليكبر مع نصّه
`%`	**نسبة من الأب**	العروض داخل حاوية
`vh · vw`	**نسبة من الشاشة**	الأقسام التي تملأ الشاشة كاملة

قاعدة مبسّطة: استعمل rem للخطوط والمسافات، و px للحدود، و % للعروض. هذا يغطّي أغلب الحالات.

المحور الرابع

التدفّق والمواضع

كيف يترتّب العنصر في الصفحة قبل أن نحرّكه



الخاصية display

block

يأخذ سطرًا كاملًا

`div · p · h1 · section`

يقبل العرض والارتفاع

inline

يأخذ عرض محتواه فقط

`span · a · strong`

لا يقبل العرض والارتفاع

inline-block

في السطر لكنه يقبل الأبعاد

`الأزرار والبطاقات الصغيرة`

حلّ وسط بين الاثنين

none

يختفي تمامًا

`القوائم المنسدلة المغلقة`

لا يحجز مكانًا في الصفحة

الفرق بين block و inline هو أول ما يربك المبتدئ. جرّبه بنفسك على span و div الآن.

الخاصية position

`static`	**الوضع الافتراضي**	يتبع تدفّق الصفحة، ولا يتأثّر بـ top و left
`relative`	**بالنسبة لنفسه**	يتحرّك قليلًا لكنه يحتفظ بمكانه الأصلي
`absolute`	**بالنسبة لأقرب أب متموضع**	يخرج من التدفّق تمامًا — للشارات والأيقونات
`fixed`	**بالنسبة للشاشة**	يبقى ثابتًا عند التمرير — للترويسة الملتصقة
`sticky`	**يلتصق عند حدّ معيّن**	عادي ثم يثبت عند التمرير — عناوين الأقسام

لا تبنِ تخطيط الصفحة بـ absolute. استعمله للتفاصيل الصغيرة فقط — التخطيط عمل Flexbox.

المحور الخامس

التخطيط بـ Flexbox

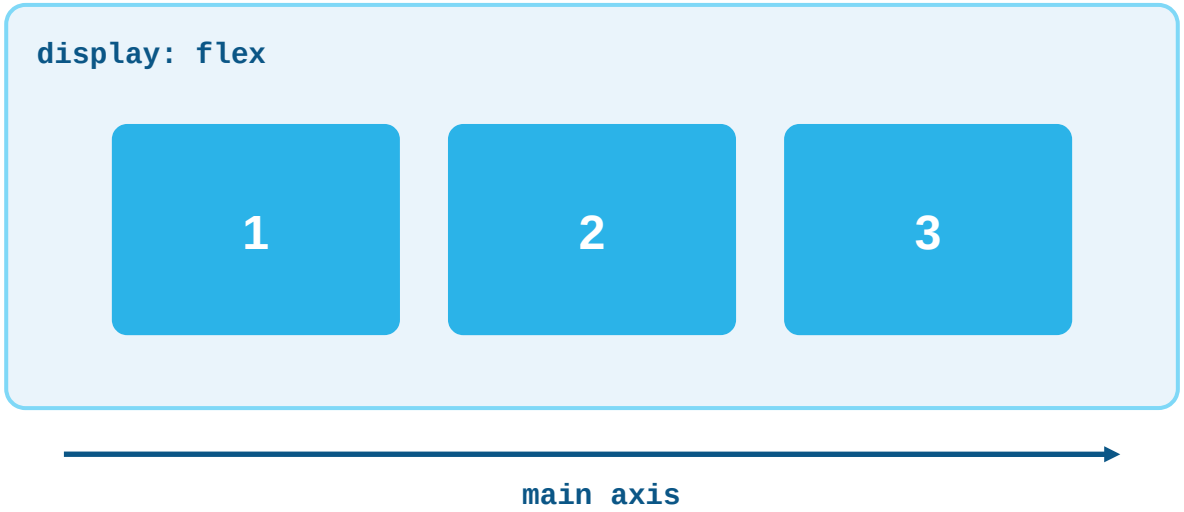
نصف الجلسة هنا — وهو أهم ما تخرج به اليوم



فكرة Flexbox

المبدأ في سطرين

تكتب display: flex على الحاوية الأب، فيصطفّ أبناؤها في صفّ واحد .ثم تتحكّم بتوزيعهم على المحور الرئيسي، ومحاذاتهم على المحور المتقاطع.




```
.container {
  display: flex;
}
```

الخصائص تنقسم قسمين: ما يُكتب على الحاوية، وما يُكتب على العناصر .لا تخلط بينهما.

خصائص الحاوية

`flex-direction`	اتجاه الصف	`row · column`
`justify-content`	التوزيع على المحور الرئيسي	`center · space-between`
`align-items`	المحاذاة على المحور المتقاطع	`center · stretch`
`flex-wrap`	السماح بالنزول لسطر جديد	`wrap · nowrap`
`gap`	المسافة بين العناصر	`gap: 1rem`

```
.row { display:flex; justify-content:space-between; align-items:center; gap:1rem; }
```

خصائص العناصر

ثلاث خصائص فقط

`flex-grow`

كم يتمدّد العنصر إن بقي فراغ

`flex-shrink`

كم ينكمش إن ضاق المكان

`flex-basis`

حجمه الأساسي قبل التمدّد

التوسيط المثالي

```
.box {
  display: flex;
  justify-content: center;
  align-items: center;
}
```

ثلاثة أسطر تحلّ مشكلة أرّقت المطوّرين سنوات: توسيط عنصر أفقيًا وعموديًا في آن.

```
/* shorthand */   flex: 1;      =      flex: 1 1 0;
```

لن تحفظ هذه الخصائص بالقراءة. أنجز Flexbox Froggy كاملة الليلة، وستحفظ نفسها.

لمحة عن Grid

الشبكة Grid

```
.grid {
  display: grid;
  grid-template-columns:
    repeat(3, 1fr);
  gap: 1rem;
}
```

الفرق في جملة

الأداة Flexbox تعمل على محور واحد: صفّ أو عمود. أمّا Grid فتعمل على محورين معًا: صفوف وأعمدة في وقت واحد.

الوحدة fr تعني حصة من المساحة المتاحة. ثلاثة أعمدة بـ 1fr تتقاسم العرض بالتساوي.

استعمل Flexbox

شريط تنقّل، صفّ أزرار، توسيط عنصر، بطاقة داخلها صورة ونص

استعمل Grid

معرض صور، تخطيط الصفحة كاملة، جدول بطاقات منتظم

في هذه الدورة: أتقن Flexbox أولًا. الأداة Grid تكفيك منها هذه اللمحة حتى تحتاجها فعلًا.

التصميم المتجاوب

لماذا نبدأ من الهاتف

- أغلب زوّارك على الهاتف فعلًا
- التصميم الضيّق أبسط، والتوسيع أسهل من التضييق
- يعني: من هذا العرض وما فوق الشرط min-width
- لا تلاحق أجهزة بعينها — غيّر عند انكسار التصميم

```
/* mobile first */
.card { width: 100%; }

@media (min-width: 768px) {
  .card { width: 50%; }
}

@media (min-width: 1200px) {
  .card { width: 33%; }
}
```

inside <head> — without it nothing responds

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

جرّب الآن: افتح F12 واضغط أيقونة الهاتف، وغيّر عرض الشاشة وأنت تنظر لصفحتك.

متغيّرات CSS

```
:root {
  --brand:   #0B5686;
  --accent:  #2BB3E8;
  --radius:  8px;
}

.btn {
  background: var(--brand);
  border-radius: var(--radius);
}
```

لماذا تهمّنا الآن

- لون العلامة التجارية يُكتب مرة واحدة
- تغيير اللون في سطر واحد يغيّر الموقع كله
- هي أساس الوضع الليلي والثيمات
- لاحقًا React ستقابل الفكرة نفسها في

ابدأ كل مشروع بكتلة root فيها ألوانك ومسافاتك. أنت تشكر نفسك بعد أسبوع.

أخطاء شائعة

نسيان box-sizing	صناديق أعرض ممّا خطّطت وتخطيط ينكسر
الإفراط في important	تكسر نظام الأولوية ولا تعرف بعدها من يفوز
بناء التخطيط بـ absolute	ينهار عند أي تغيير في المحتوى أو الشاشة
نسيان وسم viewport	الصفحة لا تستجيب على الهاتف مهما كتبت
أسماء أصناف بلا معنى	الصنف box2 لا يخبرك بشيء بعد أسبوع
التنسيق داخل وسوم HTML	يخلط البنية بالشكل ويصعّب كل تعديل لاحق
ملاحقة أجهزة بعينها	غيّر عند انكسار التصميم، لا عند مقاس هاتف معيّن

التكليف الذاتي والواجب

قراءة

مرجع W3Schools

- Selectors · Box Model
- Display · Position
- Flexbox · Grid
- Media Queries
- Colors · Fonts · Units

إلزامي

ألعاب التخطيط

- كاملة Flexbox Froggy أنجز
- كاملة Grid Garden أنجز
- لا تتوقّف عند منتصفها
- هذا ليس اقتراحًا

تطبيق

صفحة السيرة الذاتية

- نسّق صفحة الجلسة الماضية
- منفصل style.css استعمل ملف
- Flexbox اجعل شريط التنقّل بـ
- اجعلها متجاوبة على الهاتف
- استعمل متغيّرات للألوان

الجلسة القادمة نبدأ جافاسكربت — وهي المرحلة التي تُقرّر نجاحك في React.



GetAlphaBit

خلاصة الجلسة الثانية

كل عنصر صندوق

والحدود والحشو جزء من عرضه

الأداة Flexbox هي التخطيط

وما عداها تفاصيل

الألعاب ليست ترفيهًا

هي الطريقة الوحيدة لترسيخ التخطيط